

# Web Programming: HTML · CSS · JS Class Notes

Sep 30, 2026 · @Here is the code for hand fingers detection and tracking using g

## Overview

This guide builds one GitHub Pages site demonstrating everything taught in Web Programming K0126002 — HTML, CSS, and JavaScript — organized as a single repo with a master index page linking to all weekly tasks.

### Who this is for

| Situation                                                 | Where to start                               |
|-----------------------------------------------------------|----------------------------------------------|
| No GitHub repo yet                                        | Part 1, Step 1                               |
| Have a repo, cannot deploy to Pages                       | Part 1, Step 5                               |
| Deployed with AI/friend's help, don't fully understand it | Read all parts — focus on the code examples  |
| Everything already working                                | Jump to Part 5 for the master page structure |

### What you will have by the end

A repo named `webpro` (or `Web-Programming`) live at `https://username.github.io/webpro/`. The master `index.html` is a clean demo portfolio linking to every subfolder and task, organized by week.

### How this guide is organized

| Part | Topic                | You build                                         |
|------|----------------------|---------------------------------------------------|
| 1    | GitHub + Pages setup | Repo → push → live URL                            |
| 2    | HTML                 | Structure: headings, links, images, tables, forms |
| 3    | CSS                  | Style: colors, fonts, flexbox, responsive layout  |
| 4    | JavaScript           | Logic: events, DOM, functions, local storage      |

| Part | Topic          | You build                                                         |
|------|----------------|-------------------------------------------------------------------|
| 5    | Master page    | <code>index.html</code> that links to all tasks with descriptions |
| 6    | Repo structure | How to name folders and files so Pages serves them                |

## Part 1 — GitHub Setup and GitHub Pages

### Step 1: Create a GitHub account

1. Go to [github.com](https://github.com) and click **Sign up**
2. Choose a username you are comfortable sharing (students will see it in your URL)
3. Verify your email address

### Step 2: Create the repo

1. Click the + button (top right) → **New repository**
2. Repository name: `webpro` (or `Web-Programming`)
3. Set to **Public** (required for free GitHub Pages)
4. Check **Add a README file**
5. Click **Create repository**

### Step 3: Create your first file on GitHub

1. Inside your new repo, click **Add file** → **Create new file**
2. Name it `index.html`
3. Paste this into the editor:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>My Web Portfolio</title>
</head>
<body>
  <h1>Hello, World!</h1>
  <p>This is my web programming portfolio.</p>
</body>
</html>
```

4. Scroll down → click **Commit changes** → **Commit directly to the main branch** → **Commit changes**

## Step 4: Enable GitHub Pages

1. In your repo, click **Settings** (top tab)
2. In the left sidebar, click **Pages**
3. Under **Source**, select **Deploy from a branch**
4. Branch: **main** / folder: **/ (root)**
5. Click **Save**
6. Wait 1-2 minutes. Refresh the page — you will see:

Your site is live at <https://username.github.io/webpro/>

## Step 5: Install Git on your computer (for local editing)

1. Download Git: [git-scm.com/downloads](https://git-scm.com/downloads)
2. Install with default settings
3. Open a terminal (or Git Bash on Windows) and verify:

```
git --version
# git version 2.x.x
```

## Step 6: Clone the repo to your computer

```
# Replace USERNAME and REPO with yours
git clone https://github.com/USERNAME/webpro.git
cd webpro
```

You now have a local copy of your repo. Edit files with VS Code (or any editor), then push:

```
git add .
git commit -m "Add my first page"
git push
```

GitHub Pages updates automatically within 1–2 minutes after every push.

**Common mistake:** Your repo must be **Public** and the file must be named exactly `index.html` (lowercase) for Pages to serve it at the root URL.

## Part 2 — HTML Fundamentals

HTML defines the **structure** of a page. Every element is a tag: `<tag>content</tag>`.

The page skeleton (always start here)

```
<!DOCTYPE html>           <!-- Tells the browser: this is HTML5 -->
<html lang="en">           <!-- Root element -->
<head>
  <meta charset="UTF-8"> <!-- Support all languages -->
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Page Title</title> <!-- Shown in the browser tab -->
  <link rel="stylesheet" href="style.css"> <!-- Link your CSS file -->
</head>
<body>
  <!-- Everything visible goes here -->
</body>
</html>
```

## Headings and paragraphs

```
<h1>Main Title</h1>      <!-- Biggest, only one per page -->
<h2>Section Title</h2>    <!-- Sub-sections -->
<h3>Sub-section</h3>
<p>This is a paragraph. HTML ignores extra spaces and line breaks.</p>
```

## Links and images

```
<!-- Link to another page -->
<a href="https://google.com">Visit Google</a>

<!-- Link to a file in your repo (relative path) -->
<a href="week2/calculator.html">Week 2: Calculator</a>

<!-- Image -->

<!-- alt= is required: it shows when the image fails to load -->
```

## Lists

```
<!-- Unordered (bullets) -->
<ul>
  <li>HTML</li>
  <li>CSS</li>
  <li>JavaScript</li>
</ul>

<!-- Ordered (numbered) -->
<ol>
  <li>Create repo</li>
  <li>Write code</li>
  <li>Push to GitHub</li>
</ol>
```

## Table

```
<table border="1">
  <thead>
    <tr>
      <th>Name</th>
      <th>Score</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <td>Alice</td>
      <td>95</td>
    </tr>
    <tr>
      <td>Bob</td>
      <td>88</td>
    </tr>
  </tbody>
</table>
```

## Form (user input)

```
<form>
  <label for="name">Your name:</label>
  <input type="text" id="name" placeholder="Enter name">

  <label for="email">Email:</label>
  <input type="email" id="email">

  <button type="submit">Submit</button>
</form>
<!-- type="email" validates the format automatically -->
```

## Grouping elements

```
<div>  <!-- Block container: stacks vertically (for layout) -->
  <p>Inside a div</p>
</div>

<span>  <!-- Inline container: stays in the line (for styling words) -->
  <p>This word is <span style="color:red">red</span>.</p>
</span>
```

**Rule to remember:** If it's a structure or layout block → use `<div>` . If it's part of a sentence → use `<span>` .

Save as `week1/html-basics.html` in your repo

## Part 3 — CSS Styling

CSS controls how elements **look**: color, size, spacing, fonts, and layout. Link it from HTML with `<link rel="stylesheet" href="style.css">` .

### Selectors — how you target elements

```
/* Element selector - affects ALL <p> tags */
p {
  color: navy;
}

/* Class selector - use class="card" in HTML */
.card {
  background: #f0f0f0;
  border-radius: 8px;
}

/* ID selector - use id="header" in HTML (only one element) */
#header {
  font-size: 32px;
}
```

## Colors and fonts

```
body {
  /* Background color (name, hex, or rgb) */
  background-color: #ffffff;      /* white */
  color: #1a1a1a;                /* near-black text */

  /* Font: always list a fallback */
  font-family: 'Segoe UI', Arial, sans-serif;
  font-size: 16px;
  line-height: 1.6;              /* comfortable reading spacing */
}

h1 { color: #1a56db; }           /* blue heading */
a { color: #16a34a; }           /* green links */
```

## Spacing: margin and padding

```
+-----+
| margin (outside)      |
| +-----+            |
| | padding (inside)   | |
| | +-----+          | |
| | | content          | | |
| | +-----+          | |
| +-----+            |
+-----+
```

```
.card {
  margin: 20px;           /* space OUTSIDE the element */
  padding: 16px;          /* space INSIDE the element */
  border: 1px solid #ccc;
  border-radius: 8px;     /* rounded corners */
  box-shadow: 0 2px 8px rgba(0,0,0,0.1); /* subtle shadow */
}
```

## Flexbox layout (most useful layout tool)

```
/* Parent: controls HOW children are arranged */
.container {
  display: flex;
  gap: 16px;           /* space between children */
  flex-wrap: wrap;     /* items wrap to the next row on small screens */
  justify-content: center; /* center horizontally */
  align-items: center;  /* center vertically */
}

/* Child: controls HOW MUCH space each child takes */
.card {
  flex: 1;             /* grow to fill available space equally */
  min-width: 200px;    /* but never narrower than 200px */
}
```

```
<!-- HTML to go with the CSS above -->
<div class="container">
  <div class="card">Week 1</div>
  <div class="card">Week 2</div>
  <div class="card">Week 3</div>
</div>
```

## Responsive design (mobile-friendly)

```
/* Default: desktop layout, 3 columns */
.container {
  display: flex;
  gap: 16px;
}
.card { flex: 1; }

/* When screen width < 600px: stack into one column */
@media (max-width: 600px) {
  .container { flex-direction: column; }
}
```

## A complete `style.css` starter

```
* { box-sizing: border-box; margin: 0; padding: 0; }

body {
  font-family: 'Segoe UI', Arial, sans-serif;
  background: #f5f7ff;
  color: #1a1a2e;
  padding: 20px;
}

nav a {
  display: inline-block;
  margin-right: 12px;
  color: #1a56db;
  text-decoration: none;
  font-weight: bold;
}
nav a:hover { text-decoration: underline; }

.card {
  background: white;
  border: 1px solid #ddd;
  border-radius: 10px;
  padding: 20px;
  margin: 12px 0;
  box-shadow: 0 2px 6px rgba(0,0,0,0.08);
}

button {
  background: #1a56db;
  color: white;
  border: none;
  padding: 8px 18px;
  border-radius: 6px;
  cursor: pointer;
  font-size: 14px;
}
button:hover { background: #1446b5; }
```

**Save as `week2/css-basics.html`** (or add a `<style>` block inside any `.html` file to practice inline)

## Part 4 — JavaScript Basics

JavaScript makes pages **interactive**: respond to clicks, update content, store data.

Add it inside `<script>` at the bottom of `<body>`, or in a separate `script.js` file.

### Variables and output

```
// Two modern ways to declare variables
let name = 'Alice';           // can be reassigned
const MAX = 100;              // cannot be reassigned (constant)

console.log(name);            // prints to browser DevTools console (F12)
alert('Hello, ' + name);      // popup dialog
```

### Getting and changing elements on the page

```
<!-- HTML -->
<p id="message">Original text</p>
<button onclick="changeText()">Click me</button>
```

```
// JavaScript
function changeText() {
  // Get element by its id
  let el = document.getElementById('message');

  // Change what it displays
  el.textContent = 'Text was changed!';

  // Change its style
  el.style.color = 'red';
}
```

### Reading user input

```
<input type="text" id="nameInput" placeholder="Your name">
<button onclick="greet()">Greet</button>
<p id="output"></p>
```

```
function greet() {  
  let name = document.getElementById('nameInput').value;  
  document.getElementById('output').textContent = 'Hello, ' + name + '!';  
}
```

## Conditions (if/else)

```
let score = 75;  
  
if (score >= 90) {  
  console.log('A');  
} else if (score >= 80) {  
  console.log('B');  
} else if (score >= 70) {  
  console.log('C');  
} else {  
  console.log('F');  
}
```

## Loops

```
// Repeat 5 times  
for (let i = 1; i <= 5; i++) {  
  console.log('Step ' + i);  
}  
  
// Loop through an array  
let fruits = ['apple', 'banana', 'cherry'];  
for (let fruit of fruits) {  
  console.log(fruit);  
}
```

## A complete mini-app: counter

```
<!DOCTYPE html>  
<html lang="en">  
<head><meta charset="UTF-8"><title>Counter</title></head>  
<body>  
  <h2>Counter: <span id="count">0</span></h2>  
  <button onclick="change(1)">+ Add</button>
```

```

<button onclick="change(-1)">- Subtract</button>
<button onclick="reset()">Reset</button>

<script>
  let count = 0;

  function change(amount) {
    count = count + amount;
    document.getElementById('count').textContent = count;
  }

  function reset() {
    count = 0;
    document.getElementById('count').textContent = 0;
  }
</script>
</body>
</html>

```

## Saving data with localStorage

```

// Save (data persists even when page refreshes)
localStorage.setItem('username', 'Alice');

// Load
let saved = localStorage.getItem('username'); // 'Alice'

// Delete
localStorage.removeItem('username');

```

**Debug tip:** Press **F12** in your browser to open DevTools. The **Console** tab shows errors and `console.log()` output. This is your most important tool.

**Save as** `week3/js-basics.html` and `week3/counter.html` in your repo

## Part 5 — Master index.html

The root `index.html` is the first page visitors see at `username.github.io/webpro/`. Make it a clean portfolio that links to every task. Copy and adapt this template:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Web Programming Portfolio</title>
  <style>
    * { box-sizing: border-box; margin: 0; padding: 0; }
    body {
      font-family: 'Segoe UI', Arial, sans-serif;
      background: #f0f4ff;
      color: #1a1a2e;
      padding: 20px;
    }
    header {
      background: linear-gradient(135deg, #1a56db, #7c3aed);
      color: white;
      padding: 32px 24px;
      border-radius: 12px;
      margin-bottom: 28px;
    }
    header h1 { font-size: 2rem; margin-bottom: 6px; }
    header p { opacity: 0.8; font-size: 1rem; }

    .week-section { margin-bottom: 28px; }
    .week-section h2 {
      font-size: 1.2rem;
      color: #1a56db;
      margin-bottom: 12px;
      border-bottom: 2px solid #c8d4f0;
      padding-bottom: 4px;
    }

    .tasks {
      display: flex;
      flex-wrap: wrap;
      gap: 12px;
    }
    .task-card {
      background: white;
      border: 1.5px solid #c8d4f0;
      border-radius: 10px;
      padding: 16px 20px;
    }
```

```

    text-decoration: none;
    color: #1a1a2e;
    width: 200px;
    transition: box-shadow 0.2s, transform 0.2s;
}
.task-card:hover {
    box-shadow: 0 4px 16px rgba(26, 86, 219, 0.15);
    transform: translateY(-2px);
}
.task-card .tag {
    font-size: 10px;
    font-weight: bold;
    text-transform: uppercase;
    color: #1a56db;
    margin-bottom: 6px;
}
.task-card h3 { font-size: 15px; }
.task-card p { font-size: 12px; color: #666; margin-top: 4px; }

footer {
    margin-top: 40px;
    text-align: center;
    font-size: 12px;
    color: #888;
}
</style>
</head>
<body>

<header>
    <h1>🌐 My Web Programming Portfolio</h1>
    <p>Student Name · ID: 20XXXXXXXX · Gachon University · Web Programming
K0126002</p>
</header>

<!-- ===== WEEK 1: HTML ===== -->
<section class="week-section">
    <h2>Week 1 – HTML Fundamentals</h2>
    <div class="tasks">
        <a class="task-card" href="week1/html-basics.html">
            <div class="tag">HTML</div>
            <h3>HTML Basics</h3>
            <p>Tags, headings, links, images</p>
        </a>
    </div>
</section>

```

```

    </a>
    <a class="task-card" href="week1/forms.html">
      <div class="tag">HTML</div>
      <h3>HTML Forms</h3>
      <p>Input, select, submit</p>
    </a>
  </div>
</section>

<!-- ===== WEEK 2: CSS ===== -->
<section class="week-section">
  <h2>Week 2 – CSS Styling</h2>
  <div class="tasks">
    <a class="task-card" href="week2/css-basics.html">
      <div class="tag">CSS</div>
      <h3>CSS Basics</h3>
      <p>Colors, fonts, spacing</p>
    </a>
    <a class="task-card" href="week2/flexbox.html">
      <div class="tag">CSS</div>
      <h3>Flexbox Layout</h3>
      <p>Responsive card grid</p>
    </a>
  </div>
</section>

<!-- ===== WEEK 3: JAVASCRIPT ===== -->
<section class="week-section">
  <h2>Week 3 – JavaScript</h2>
  <div class="tasks">
    <a class="task-card" href="week3/counter.html">
      <div class="tag">JS</div>
      <h3>Counter App</h3>
      <p>Click events and DOM updates</p>
    </a>
    <a class="task-card" href="week3/quiz.html">
      <div class="tag">JS</div>
      <h3>Quiz App</h3>
      <p>If/else, score tracking</p>
    </a>
    <a class="task-card" href="week3/todo.html">
      <div class="tag">JS</div>
      <h3>To-Do List</h3>

```

```

    <p>Arrays, localStorage</p>
  </a>
</div>
</section>

<footer>
  <p>Built with HTML, CSS & JavaScript · Hosted on GitHub Pages</p>
</footer>

</body>
</html>

```

**How to add a new task:** Copy one `<a class="task-card" ...>` block, update the `href` to point to your new file, and update the title and description. The card will appear automatically in the correct week section.

## Part 6 — Repo File Structure

This is the recommended layout for your `webpro` repo. Every subfolder has its own `index.html` (the default page GitHub Pages loads when you visit the folder URL).

```

webpro/                                ← root of the repo
├──
├── index.html                          ← MASTER page (links to everything)
├── style.css                           ← shared styles (optional)
├── images/
│   └── profile.jpg
├──
├── week1/                              ← Week 1 tasks
│   ├── index.html                     ← week1 overview (optional)
│   ├── html-basics.html
│   └── forms.html
├──
├── week2/                              ← Week 2 tasks
│   ├── css-basics.html
│   └── flexbox.html
├──
├── week3/                              ← Week 3 tasks
│   └── counter.html
└──

```

```

|   ├── quiz.html
|   └── todo.html
|
└── README.md

```

← shown on GitHub repo page

## How URLs map to files

| File path in repo                   | URL on GitHub Pages                                           |
|-------------------------------------|---------------------------------------------------------------|
| <code>index.html</code>             | <code>username.github.io/webpro/</code>                       |
| <code>week1/html-basics.html</code> | <code>username.github.io/webpro/week1/html-basics.html</code> |
| <code>week3/counter.html</code>     | <code>username.github.io/webpro/week3/counter.html</code>     |

## How to create a subfolder on GitHub (browser)

1. Inside your repo, click **Add file** → **Create new file**
2. In the filename box, type `week1/html-basics.html` — the `/` automatically creates the folder
3. Add your content and commit

## How to create a subfolder locally (Git)

```

mkdir week1
cd week1
# create your file, then:
cd ..
git add week1/html-basics.html
git commit -m "Add Week 1: HTML basics"
git push

```

## Naming rules (GitHub Pages is case-sensitive on Linux servers)

- Use **lowercase only**: `week1`, not `Week1`
- Use **hyphens** instead of spaces: `html-basics.html`, not `html basics.html`
- The entry-point file for a folder must be named exactly `index.html`

## Checklist before sharing your URL

- ☐ `index.html` exists in the root folder
- ☐ GitHub Pages is enabled (Settings → Pages → Source: main / root)
- ☐ Repo is set to **Public**
- ☐ At least one commit has been pushed after enabling Pages
- ☐ Wait 2 minutes, then open `https://username.github.io/webpro/`